[C4Scale] - HRMS | Fullstack Take Home Assignment. Build a Mini HRMS Module: Employee Directory + Leave Management

Objective

Evaluate your ability to design and implement a practical, production-leaning full-stack HRMS module with correct data modeling, role-based access control, and a simple approval workflow.

Scenario

You are building a mini HRMS module for an organization where:

- Employees can view an employee directory and apply for leave.
- Managers can review and approve/reject leave requests for their direct reports.
- HR Admins can view all leave requests and employee records.
- The UI should clearly reflect role-based permissions and request status updates.

This is intentionally a mini-module (not a full HRMS). Focus on correctness, clarity, and good engineering tradeoffs.

Requirements

Backend (choose ONE): **Node.js** (Express / NestJS) **or Python** (FastAPI)

Core expectations

- REST API (preferred) or GraphQL (optional)
- DB persistence using:
    - **PostgreSQL** (preferred), **or** SQLite (acceptable)
- Authentication + Authorization:Simple auth is fine (JWT/session): **Backend-enforced Role based authorization is mandatory** (frontend hiding buttons is not enough)

Suggested Data Model

Employee: `id`, `name`, `email`, `department`, `role` (employee | manager | hr_admin), `manager_id` (nullable; points to another Employee)

LeaveRequest: `id`, `employee_id`, `start_date`, `end_date`, `leave_type` (e.g., annual | sick | unpaid), `reason` (optional), `status` (submitted | approved | rejected), `reviewed_by` (nullable; manager/hr id), `review_comment` (optional), `created_at`, `updated_at`

Audit Log (minimum): `id`, `entity_type` (e.g., "leave_request"), `entity_id`, `action` (created | approved | rejected | updated), `performed_by`, `performed_at`, `metadata` (optional JSON)

Business Rules

Implement **ONE of the following** with tests:

1. **No overlap rule:** an employee cannot submit a leave request that overlaps another submitted/approved request.
2. **Manager constraint:** only the assigned manager can approve/reject leave requests for direct reports.
3. **Leave balance (simple):** employee has an integer `annual_leave_balance`, and annual leave cannot exceed it.

Pick one and state it in the README as the chosen rule.

API Scope (minimum endpoints)

Auth (or mocked auth)

- Login and role identification, or a clear mock approach (e.g., “pass `x-user-id` header”)

Employees

- `Get employees`

  `Seeded Data in the DB is fine`

Leaves

- `Manage leaves`

Audit

- Create audit entries for: create, approve, reject (at minimum)

Frontend (choose ONE)

- **React.js** or **Next.js**

UI Views (minimum)

1) Employee Directory

- List employees with basic fields (name, department, role)
- Click-through to profile page/view

2) Leave Requests

- Employee:
  - “Apply for leave” form
  - “My leave requests” list with status
- Manager:
  - “Pending approvals” list
  - Approve / reject with optional comment

- HR Admin:
    - “All leave requests” list
    - Filter by status (basic)

UI Expectations

- Clear status indicators (submitted/approved/rejected)
- Role-based navigation and controls
- Clean, usable layout (no need for a full design system)

Nice-to-Have (Bonus)

Only attempt these if you have time after completing the must-haves.

- Docker Compose for app + DB
- Stronger test coverage (integration tests)
- Deployment (Vercel + Render/Fly/etc.) and a live URL

Timeline

- Aim for **~6–10 hours total effort**
- Submit within **3 days** of receiving the assignment (unless otherwise agreed)

Evaluation Criteria

- Clear API design, RBAC enforced server-side, correctness of workflow; Clean schema, sensible relationships; Usable UI, role-based views, good state handling
- Leave submission → manager approval/rejection → reflected in UI; RBAC + one business rule tested
- Thoughtful tradeoffs; bonuses if time permits; README clarity, setup simplicity, assumptions documented

Submission

Share:

- **A recorded Loom (or other) Video with the link is mandatory**. Record your voice over while sharing the screen and explaining your code. **Demonstrate:**
    - Employee Screen and creating a leave request
    - Approving/rejecting as manager
    - HR Admin viewing all requests
- **Zip the entire code and upload it to a google drive link. Share this link**
  (OR)
  Provide a GitHub repository (PRIVATE)
    - Source code for frontend + backend **and Clear README** including:
        - Setup instructions (local run) and Your chosen business rule (1 of the 3) and how you implemented it

We want to see your system design thinking, your coding prowess and importantly your passion in building software! Good Luck !